NAME – SUBASH G
REG NO – 192572150

COURSE NAME – Operation System
COURSE CODE – CSA0408

Lab Observations

EXPERIMENT - 7

Construct a C program to implement a non-preemptive SJF algorithm.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int at[10], bt[10], pr[10];
    int n, i, j, temp;
    int time = 0, count, over = 0;
    int sum_wait = 0, sum_turnaround = 0, start;
    float avgwait, avgturn;

    printf("Enter the number of processes\n");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter the arrival time and execution time for process
%d\n", i + 1);
        scanf("%d%d", &at[i], &bt[i]);

        pr[i] = i + 1;
    }

    /* Sort processes according to arrival time */
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
```

```
        {
            if(at[i] > at[j])
            {
                temp = at[i];
                at[i] = at[j];
                at[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
            }
        }
    }

    printf("\n\nProcess\t|Arrival time\t|Execution time\t|Start
time\t|End time\t|Waiting time\t|Turnaround time\n\n");

    while(over < n)
    {
        count = 0;

        /* Find processes that have arrived */
        for(i = over; i < n; i++)
        {
            if(at[i] <= time)
                count++;
            else
                break;
        }

        /* Select process with shortest burst time */
        if(count > 1)
        {
            for(i = over; i < over + count - 1; i++)
            {
                for(j = i + 1; j < over + count; j++)
                {
                    if(bt[i] > bt[j])
                    {
```

```
                    temp = at[i];
                    at[i] = at[j];
                    at[j] = temp;

                    temp = bt[i];
                    bt[i] = bt[j];
                    bt[j] = temp;

                    temp = pr[i];
                    pr[i] = pr[j];
                    pr[j] = temp;
                }
            }
        }
    }

    /* Execute selected process */
    start = time;

    time = time + bt[over];

    printf("P[%d]\t|\t%d\t|\t%d\t|\t%d\t|\t%d\t|\t%d\t|\t%d\n",
           pr[over],
           at[over],
           bt[over],
           start,
           time,
           time - at[over] - bt[over],
           time - at[over]);

    sum_wait += time - at[over] - bt[over];

    sum_turnaround += time - at[over];

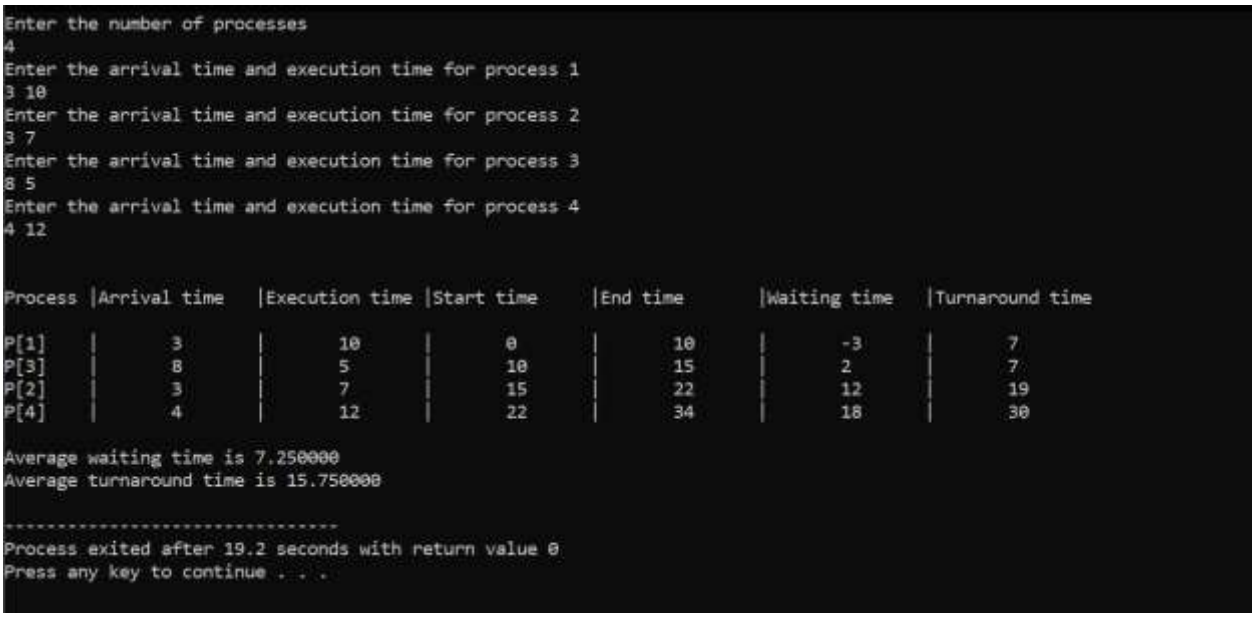
    over++;
}

avgwait = (float)sum_wait / n;
avgturn = (float)sum_turnaround / n;

printf("\nAverage waiting time is %f\n", avgwait);
printf("Average turnaround time is %f\n", avgturn);
```

```
    return 0;
}
```

OUTPUT:



```
Enter the number of processes
4
Enter the arrival time and execution time for process 1
3 10
Enter the arrival time and execution time for process 2
3 7
Enter the arrival time and execution time for process 3
8 5
Enter the arrival time and execution time for process 4
4 12


Process |Arrival time   |Execution time |Start time     |End time       |Waiting time   |Turnaround time

P[1]    |       3       |       10      |       0       |       10      |       -3      |       7
P[3]    |       8       |       5       |       10      |       15      |       2       |       7
P[2]    |       3       |       7       |       15      |       22      |       12      |       19
P[4]    |       4       |       12      |       22      |       34      |       18      |       30

Average waiting time is 7.250000
Average turnaround time is 15.750000

--------------------------------
Process exited after 19.2 seconds with return value 0
Press any key to continue . . .
```